



北京師範大學
BEIJING NORMAL UNIVERSITY

《计算机图形学》课程实验

实验八：质点-弹簧系统

授课教师：张鸿文

助教：李玉慧

课程网页：<https://zhanghongwen.cn/cg>

实验八：质点-弹簧系统

一、 实验介绍

1. 教程与相关链接

实验八教程

https://docs.taichi-lang.cn/docs/cloth_simulation

环境配置教程

<https://docs.taichi-lang.cn/docs/install>

注意：云盘压缩包中包含示例代码、参考中文注释、参考论文

2. MassSpring 环境配置

Anaconda / Miniconda

<https://www.anaconda.com/download>

taichi 库：打开 Anaconda PowerShell Prompt，运行下列命令，配置 MassSpring 环境
(此处用“#”表示注释开始，在复制命令时**不要复制注释**)

https://docs.taichi-lang.cn/docs/hello_world

```
conda init

conda create -n MassSpring python=3.9
conda activate MassSpring

pip install taichi # 二选一，从默认的 pypi 源安装
pip install taichi -i https://pypi.douban.com/simple # 二选一，从豆瓣镜像源安装
```

3. 测试环境

运行示例代码“mass_spring_3d.py”后，可以在窗口中查看固定视角下实时渲染的动态场景。

https://docs.taichi-lang.cn/docs/cloth_simulation%E6%BA%90%E4%BB%A3%E7%A0%81

运行示例代码“mass_spring_2d.py”，可以在窗口中查看实时渲染的二维动态场景。

<https://github.com/taichi->

[dev/taichi/blob/master/python/taichi/examples/simulation/implicit_mass_spring.py](https://github.com/taichi-dev/taichi/blob/master/python/taichi/examples/simulation/implicit_mass_spring.py)

```
conda activate MassSpring

D: # 与下一步 cd 命令的磁盘号对应
cd "D:\projects\CG_E8_data"
python mass_spring_3d.py
# python mass_spring_3d_offline.py
python mass_spring_2d.py
# python mass_spring_2d_offline.py
```

注意：示例代码“mass_spring_3d_offline.py”、“mass_spring_2d_offline.py”中核心代码不变，运行后可以在“\results”文件夹下查看渲染的图像结果，方便保存渲染图像。

二、实验任务

1. 实验要求

1. **要求 1：**理解动态场景渲染的基本流程，了解用 Taichi 框架更新和渲染场景的方法；
- 要求 2：**学习质点-弹簧系统原理，比较三种数值积分方法（显式/隐式/半隐式欧拉）的区别；
- 要求 3：**编写代码实现不同数值积分方法的物理模拟，并展示渲染效果。

2. 具体任务

任务 1：

在“mass_spring_3d.py”的基础上，修改质点网格大小 n 、模拟速度 N ，并运行代码。
(至少设置三个不同尺度的网格大小，如 $n=8, 32, 128$)

结合理论课第 13 讲，分析代码中的 substep 函数中每一帧的更新逻辑，在**实验文档**中回答并解释：

- (1) “mass_spring_3d.py”所用的数值积分方法是哪种？
- (2) 时间步长较大 (N 较小) 时，该方法是否能收敛？（无法收敛表现为窗口闪烁）
- (3) 如果无法收敛，测试并给出不同质点网格大小 n 对应的最大时间步长 dt 的估计值。
- (4) 结合代码，在任一渲染图像上画图分析任一质点的受力情况（不需要标注具体数值，弹力画一个即可）。

任务 2：

在“mass_spring_2d.py”的基础上，修改质点网格大小 N 、时间步长 h ，并运行代码。
(至少设置三个不同尺度的网格大小，如 $n=8, 32, 128$)

结合理论课第 13 讲，分析代码中的 update 函数及相关子函数中每一帧的更新逻辑，在**实验文档**中回答并解释：

- (1) “mass_spring_2d.py”所用的数值积分方法是哪种？
- (2) 时间步长较大 (h 较大) 时，该方法是否能收敛？（无法收敛表现为窗口闪烁）
- (3) 如果无法收敛，测试并给出不同质点网格大小 n 对应的最大时间步长 dt 的估计值。
- (4) 结合 compute_force 函数，在任一渲染图像上画图分析任一质点的受力情况（不需要标注具体数值，弹力画一个即可）。

任务 3：

在“mass_spring_2d.py”的基础上，修改 update 及相应函数，实现数值积分方法（显式欧拉法，隐式欧拉法，半隐式欧拉法）中的另外两种；

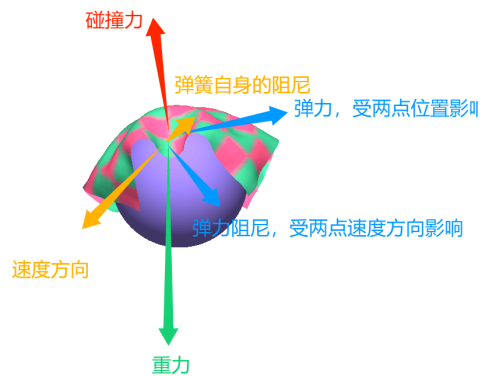
结合理论课第 13 讲，对比三种数值积分方法的运行结果，**保存三种方法的在线渲染帧率、离线渲染图像序列或录制视频**，并在**实验文档**中：

- (1) 结合公式与代码，对比关键代码，分析三种数值积分方法的更新逻辑。
- (2) 测试并给出对于相同质点网格大小 n ，三种方法下对应的最大时间步长 dt 的估计值。
- (3) 对比三种方法的实现复杂度、运行效率、模拟效果；结合运行结果，分析三种方法的优缺点。

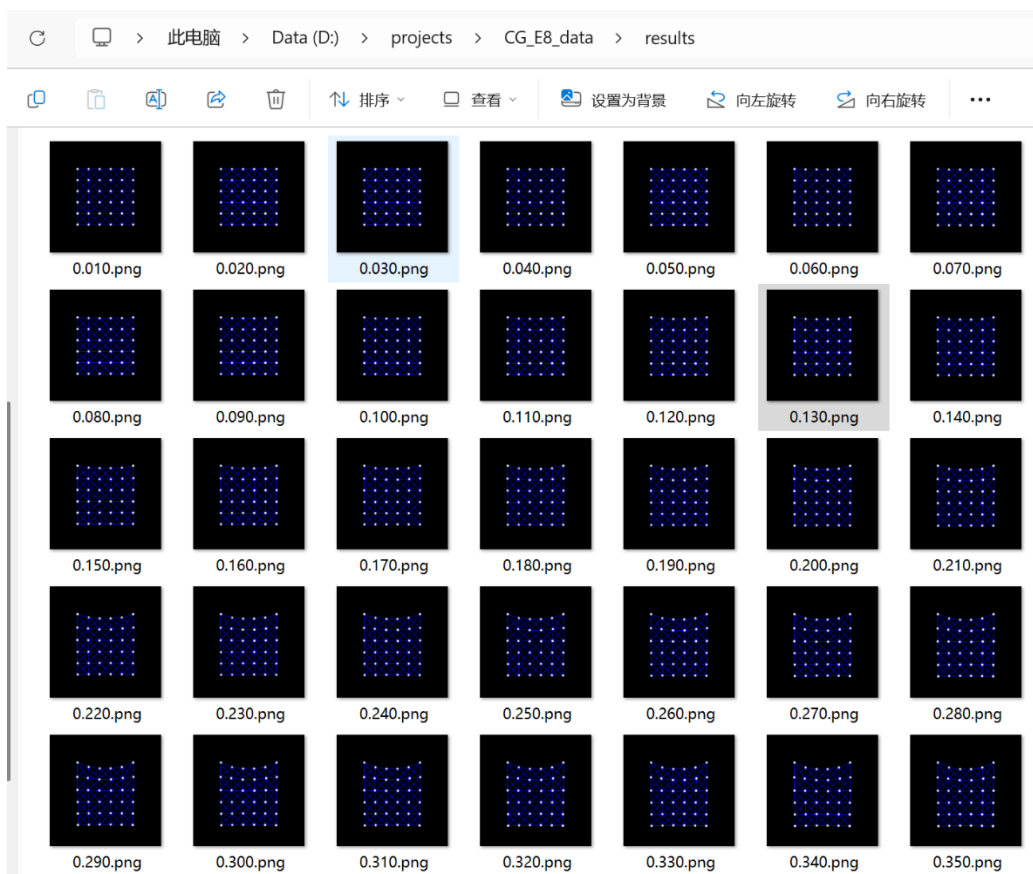
注意：任务三主要根据 `compute_force` 函数得到的每个质点的最终受力，并结合“`mass_spring_3d.py`”通过增加阻尼、设置两个固定点受力为 0 来简单修改即可；修改后另外两种方法的结果不需要与原论文方法完全一致。

3. 参考结果

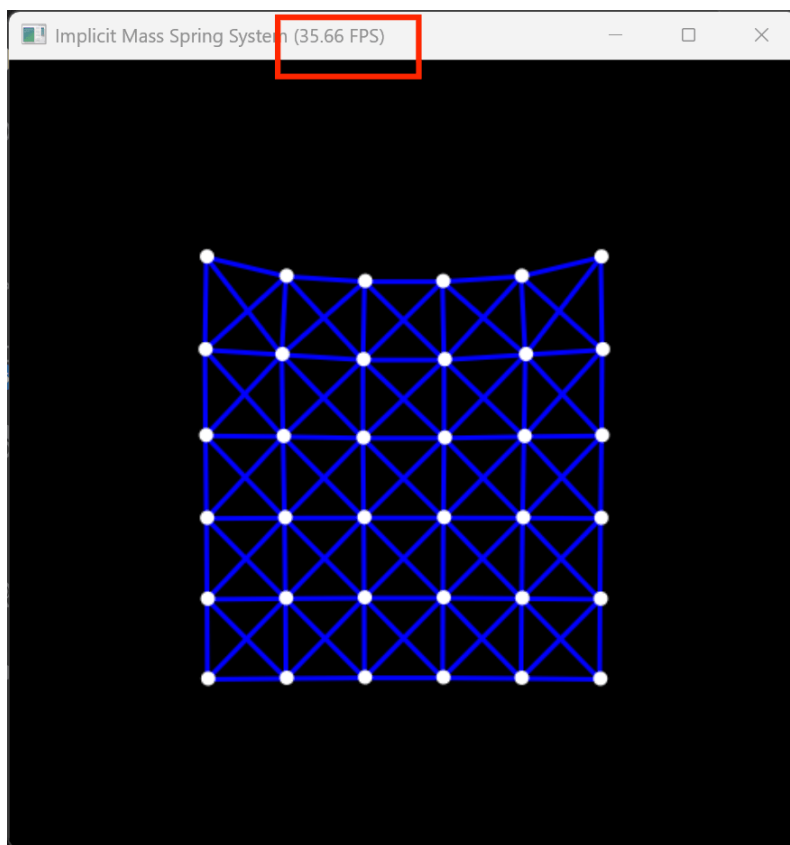
实现复杂度为代码编写的难度，根据每个方法的代码与原理分析即可。



上图为参考受力情况分析，可附加公式如 $F=mg$ ，不需要写出具体数据。



上图为离线渲染得到的渲染图像，根据肉眼观测到的模拟动态结果，对比不同方法的模拟质量，即碰撞真实度、弹性真实度等即可。



```
(MassSpring)PS D:\projects\CG_E8_data> python mass_spring_2d.py
[Taichi] version 1.7.3, llvm 15.0.1, commit 5ec301be, win, python 3.9.21
[Taichi] Starting on arch=x64
Time taken for 1.5s simulation: 4.301379 seconds
Time taken for 1.5s simulation: 4.104385 seconds
Time taken for 1.5s simulation: 4.254146 seconds
Time taken for 1.5s simulation: 4.327866 seconds
Time taken for 1.5s simulation: 4.268874 seconds
```

在线渲染得到实时帧率，同时在命令行中会输出每 1.5s 模拟时间对应的实际运行时间。根据得到的数据对比不同方法的**运行效率**，即实时渲染速度。（离线渲染需要存储中间图像结果，其运行时间不能作为参考）